

INHA UNIVERSITY TASHKENT

DEPARTMENT OF CSE & ICE

SPRING SEMESTER 2024

SOC 2040 - SYSTEMS PROGRAMMING

LAB ASSIGNMENT 1

INSTRUCTIONS :

- ALL LAB ASSIGNMENTS ARE TO BE COMPLETED IN GROUPS OF 4 TO 6 STUDENTS.
- SCREEN SHOTS ARE TO BE PROVIDED FOR ALL QUESTIONS.
- LAB ASSIGNMENT REPORT SHOULD BE PREPARED USING THE TEMPLATE PROVIDED AT THE ECLASS.
- **ONE HARDCOPY OF THE LAB ASSIGNMENT1 REPORT (TEAM LEADER'S REPORT) IN SPIRAL BOUND FORM MUST BE HANDED IN BY THE TEAM LEADER BY VISITING MY OFFICE ON WEDNESDAY 6th MARCH 2024 AFTERNOON ANYTIME BETWEEN 2PM & 5PM.**
- EVERY MEMBER OF THE TEAM MUST UPLOAD THE SOFTCOPY OF THE REPORT AT THE E-CLASS PORTAL.
- **LAST DATE FOR SUBMISSION OF THE LAB ASSIGNMENT1 REPORT AT THE ECLASS IS TUESDAY 5TH MARCH 2024**
- LATE SUBMISSIONS ARE NOT ENTERTAINED, ADHERE TO THE DEADLINE STRICTLY
- READ THE QUESTIONS CORRECTLY & CAREFULLY

ATTENTION : SYSTEM PROGRAMMING LAB **ASSIGNMENT 1 TEAM LEADERS & TEAM** **MEMBERS**

GUIDELINES FOR PREPARING LAB REPORT & **SUBMISSION**

- **EVERY MEMBER MUST PREPARE HIS/HER OWN LAB REPORT USING LAB REPORT TEMPLATE PROVIDED AT THE ECLASS.**
- **EVERY MEMBER INCLUDING THE TEAM LEADER MUST EXECUTE PART I QUESTIONS 1) & 2), PART II QUESTIONS 9) TO 14) AND PART III QUESTION 23) CONTAINING 100 COMMANDS ON HIS/HER OWN COMPUTER.**
- **EVERY MEMBER'S LAB REPORT MUST CONTAIN SCREENSHOTS FOR PART I QUESTIONS 1) & 2), FOR PART II QUESTIONS 9) TO 14) AND FOR PART III QUESTION 23) CONTAINING 100 COMMANDS EXECUTED ON HIS/HER OWN COMPUTER.**
[FILES CREATED IN PART II QUESTIONS 9) TO 14) ARE ALSO REQUIRED IN ORDER TO COMPLETE PART III-QUESTION 23]
- **REMAINING QUESTIONS CAN BE SOLVED AS A TEAM WORK (ASSIGNING TO DIFFERENT MEMBERS AND THEN COMPILING THE ANSWERS & SCREENSHOTS) AND APPROPRIATE ANSWERS & SCREENSHOTS CAN BE INCLUDED IN EVERY MEMBERS' LAB REPORT.**
- **EVERY MEMBER'S LAB REPORT MUST CONTAIN ANSWERS FOR ALL THE QUESTIONS.**
- **EVERY MEMBER MUST UPLOAD SOFCOPY OF HIS/HER REPORT AT THE E-CLASS PORTAL WELL BEFORE THE SPECIFIED DEADLINE i.e., **TUESDAY 5TH****

MARCH 2024 (WELL BEFORE MIDNIGHT OF THE SPECIFIED DATE OF SUBMISSION).

- **NO LAB ASSIGNMENT1 REPORT SUBMISSION AT THE ECLASS PORTAL WILL LEAD TO A GRADE OF 0.**

- ONE HARDCOPY OF THE LAB ASSIGNMENT1 REPORT (TEAM LEADER'S REPORT) IN SPIRAL BOUND FORM MUST BE HANDED IN BY THE TEAM LEADER BY VISITING MY OFFICE ON WEDNESDAY 6TH MARCH 2024 AFTERNOON ANY-TIME BETWEEN 2PM & 5PM.

VIRTUAL BOX & UBUNTU LINUX DOWNLOAD LINKS
YOU CAN ACCESS MY GOOGLE DRIVE TO DOWNLOAD VIRTUAL BOX & UBUNTU LINUX USING LINK GIVEN BELOW:

**TO DOWNLOAD VIRTUAL BOX 7 FOR WINDOWS HOSTS AND UBUNTU 22.04 LTS
CLICK THE FOLLOWING LINK**

https://drive.google.com/file/d/1BYR_5SZSBkcCA7qA3l8wNgf-xazxDS07/view?usp=sharing

**TO DOWNLOAD VIRTUAL BOX 7 FOR WINDOWS HOSTS AND UBUNTU 20.04 LTS
CLICK THE FOLLOWING LINK**

https://drive.google.com/file/d/1BYR_5SZSBkcCA7qA3l8wNgf-xazxDS07/view?usp=sharing

OR

**VISIT VIRTUAL BOX DOWNLOAD SITE BY CLICKING THE
FOLLOWING LINK**

<https://www.virtualbox.org/wiki/Downloads>

AND

**VISIT UBUNTU DESKTOP DOWNLOAD SITE BY CLICKING THE
FOLLOWING LINK**

<https://ubuntu.com/download/desktop>

QUESTIONS :

PART I : VIRTUAL BOX AND LINUX INSTALLATION

- 1.INSTALL THE LATEST VIRTUAL BOX ON YOUR DESKTOP/LAPTOP COMPUTER AND PROVIDE THE SCREENSHOTS OF THE COMPLETE STEP-BY-STEP INSTALLATION STEPS. INCLUDE ALL THE INSTALLATION SCREENSHOTS IN YOUR LAB REPORT AND ALSO INDICATE THE SETTINGS USED.**
- 2.INSTALL UBUNTU 20.04 LTS/ 22.04 LTS ON THE VIRTUAL BOX ON YOUR DESKTOP/LAPTOP. PROVIDE THE SCREENSHOTS OF THE COMPLETE STEP-BY-STEP INSTALLATION STEPS. INCLUDE ALL THE INSTALLATION SCREENSHOTS IN YOUR LAB REPORT AND ALSO INDICATE THE SETTINGS USED.**
- 3.What is Virtualization? Discuss the Virtual Box features in detail. **(Provide proper References)**
- 4.Explain what is meant by an Operating System. Discuss the functions performed by an operating System. List out SIX most important Operating Systems most commonly used. **(Provide proper References)**
- 5.Write short notes on Unix & Linux Operating Systems describing their history and features. **(Provide proper References)**
- 6.What is Unix/Linux Shell? Indicate which shell (default shell) you will be normally in when you login to the Linux Machine.
- 7.List out the contents of your home directory, root directory and subdirectories in root directory. **Include ALL THE SCREEN SHOTS**

PART II : USE OF EDITORS – vi, gedit

C PROGRAMMING, COMPILATION & EXECUTION ON LINUX

8. Describe the commands of Vi/Vim Editor.

9. a) Using vi Editor, create the following 'C' program file in your user directory by entering the command at the linux shell prompt : `$vi ascii.c`
- b) Compile the program using gcc - `$gcc -o ascii ascii.c` and execute the executable program - `./ascii`
- c) **PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS**
FILE SHOULD BE GIVEN THE SAME NAME AS GIVEN IN THE Program file name. THIS FILE IS REQUIRED IN PART III QUESTION 23).

//Program file name : `ascii.c`

// C Program to display all ASCII codes and their corresponding ASCII characters

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    printf("\n ASCII CODE \t ASCII CHARACTER \n ");
```

```
    printf("\n ----- \t ----- \n ");
```

```
    for(i=0; i<= 127; i++)
```

```
        printf("      %d      \t      %c      \n", i, i);
```

```
}
```

10. a) Using vi Editor, create the following 'C' program file in your user directory by entering the command at the linux shell prompt : `$vi lower2upper.c`
- b) Compile the program using gcc - `$gcc -o lower2upper lower2upper.c` and execute the executable program - `./lower2upper`
- c) **PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (YOU NEED TO**

TRY OUT ATLEAST 30 SENTENCES OF DIFFERENT CHARACTERS).

FILE SHOULD BE GIVEN THE SAME NAME AS GIVEN IN THE Program file name. THIS FILE IS REQUIRED IN PART III QUESTION 23).

//Program file name : lower2upper.c

// C Program to read a sentence from the keyboard and convert all the lowercase characters to uppercase and display the results

```
#include <stdio.h>
#include <string.h>

int main()
{
    int i, j, k, l, len;
    char ch, name[255], rname[255], uname[255], lname[255];

    printf("\n Enter your name : ");
    fgets(name, sizeof(name), stdin);

    len = strlen(name);
    printf("\n Length of your name : %d\n", --len);
    name[len]='\0';

    // Converting Lowercase characters to Uppercase

    strcpy(uname, name);
    for(i=0; i<len; i++)
    {
        if((uname[i] >= 'a') && (uname[i] <= 'z'))
            uname[i]=uname[i] - 32;
    }

    printf("\nThe given name in UPPERCASE : %s \n", uname);
}
```

- 11. a) Using vi Editor, create the following 'C' program file in your user directory by entering the command at the linux shell prompt : \$vi upper2lower.c**
- b) Compile the program using gcc - \$gcc -o upper2lower upper2lower.c and execute the executable program - \$./upper2lower**
- c) PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (YOU NEED TO TRY OUT ATLEAST 30 SENTENCES OF DIFFERENT CHARACTERS).**

FILE SHOULD BE GIVEN THE SAME NAME AS GIVEN IN THE Program file name. THIS FILE IS REQUIRED IN PART III QUESTION 23).

//Program file name : upper2lower.c

// C Program to read a sentence from the keyboard and convert all the

// uppercase characters to lowercase and display the results

#include <stdio.h>

#include <string.h>

int main()

{

int i, j, k, l, len;

char ch, name[255], rname[255], unname[255], lname[255];

printf("\n Enter your name : ");

fgets(name, sizeof(name), stdin);

len = strlen(name);

printf("\n Length of your name : %d\n", --len);

name[len]='\0';

//Converting Uppercase characters to Lowercase

strcpy(lname, name);

for(i=0; i<len; i++)

{

if((lname[i] >= 'A') && (lname[i] <= 'Z'))

lname[i]=lname[i]+32;

}

printf("\nThe given name in LOWERCASE : %s \n", lname);

}

- 12. a) Using vi Editor, create the following 'C' program file in your user directory by entering the command at the linux shell prompt : \$vi palindrome.c**
- b) Compile the program using gcc - \$gcc -o palindrome palindrome.c and execute the executable program - \$./palindrome**
- c) PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (YOU NEED TO TRY OUT ATLEAST 30 NAMES OF DIFFERENT CHARACTERS).**

FILE SHOULD BE GIVEN THE SAME NAME AS GIVEN IN THE Program file name. THIS FILE IS REQUIRED IN PART III QUESTION 23).

//Program file name : palindrome.c

// C Program to read a name from the keyboard and perform the following

// operations on the name : (i) find the length of the name (ii) reverse the name

// and (iii) check whether the given name is palindrome

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    int i, j, k, l, len;
```

```
    char ch, name[255], rname[255], unname[255], lname[255];
```

```
    printf("\n Enter your name : ");
```

```
    fgets(name, sizeof(name), stdin);
```

```
    printf("\n HELLO  %s\n", name);
```

```
    len = strlen(name);
```

```
    printf("\n Length of your name : %d\n", --len);
```

```
    name[len]='\0';
```

```
    // Reversing the name
```

```
    strcpy(rname, name);
```

```
    j=len-1;
```

```
    k=len/2;
```

```
    for(i=0; i<k; i++)
```

```
    {
```

```
        ch=rname[j];
```

```
        rname[j]=rname[i];
```

```
        rname[i]=ch;
```

```
        j--;
```

```
    }
```

```
    rname[len]='\0';
```

```
    printf("\n REVERSED NAME : %s\n",rname);
```

```
    // Checking for Palindrome
```

```
    if (!strcmp(rname, name, len))
```

```
        printf("\nThe given name is a PALINDROME \n");
```

```
    else
```

```
        printf("\nThe given name is not a PALINDROME \n");
```

```
}
```


13. a) Using **gedit** Editor, create the following file in your user directory by entering the command at the linux shell prompt : **\$gedit sum_avgl.c**

b) Compile the program using gcc (**\$gcc -o sum_avgl sum_avgl.c**) and execute the executable program(**./sum_avgl**).

c) **PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (YOU NEED TO TRY OUT ATLEAST 30 LISTS CONTAINING MORE THAN 10 NUMBERS).**

FILE SHOULD BE GIVEN THE SAME NAME AS GIVEN IN THE Program file name. THIS FILE IS REQUIRED IN PART III QUESTION 23).

//C Program file name : sum_avgl.c

**// C Program to read a list of n integers from the keyboard and
// compute the sum and average**

```
#include <stdio.h>
int main()
{
    int i, n, sum, list[100];
    float avg;

    // Read the value of n
    printf("\n Enter the size of the list : ");
    scanf("%d", &n);

    // Read the list of 'n' numbers from the keyboard
    printf("\n Enter %d Elements of the list : \n", n);
    for(i=0; i<n; i++)
    {
        printf("\n list[%d]= ", i);
        scanf("%d", &list[i]);
    }

    //Compute the sum of the list
    sum=0;
    for(i=0; i<n; i++)
        sum += list[i];

    //Compute the average of the list
    avg=(float)sum/n;
    printf("\n sum of %d elements in the list= %d and average = %f\n", n, sum, avg);
}
```

14. a) Using **gedit** Editor, create the following file in your user directory by entering the command at the linux shell prompt : **\$gedit key_search.c**
- b) Compile the program using gcc (**\$gcc -o key_search key_search.c**) and execute the executable program(**./key_search**).
- c) **PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (YOU NEED TO TRY OUT ATLEAST 30 LISTS CONTAINING MORE THAN 10 NUMBERS).**

FILE SHOULD BE GIVEN THE SAME NAME AS GIVEN IN THE Program file name. THIS FILE IS REQUIRED IN PART III QUESTION 23).

//C Program file name : key_search.c

**// C Program to read a list of n integers and a key from the keyboard and search
// for the key in the given list – If the key is found in the list then print that key is
// found and its location in the list. Also print the number of occurrences of the
// key in the list. If the key is not found in the list then print the message-
// key not found**

```
#include <stdio.h>
int main()
{
    int i, n, key, count=0, list[100];
    float avg;

    // Read the value of n
    printf("\n Enter the size of the list : ");
    scanf("%d", &n);

    // Read the list of 'n' numbers from the keyboard
    printf("\n Enter %d Elements of the list : \n", n);
    for(i=0; i<n; i++)
    {
        printf("\nlist[%d]=",i);
        scanf("%d",&list[i]);
    }

    // To search for a key in the above list
    printf("\nEnter the search key :");
    scanf("%d", &key);
    for(i=0; i<n; i++)
    {
        if(list[i]==key)
        {
```

```

        printf("\nkey %d is found at location %d in the list\n",key,i);
        count++;
    }
}
if(count==0)
    printf("\nKey %d not found in the list \n", key);
else
    printf("\nKey %d occurs %d times in the list\n", key, count);
}

```

15. Write a 'C' program to read a list of 'n' integers from the keyboard and rearrange the elements in the list **(i) in ascending order and (ii) in descending order and display the results**. The value of 'n' must be read from the keyboard at run time by giving the following prompt message to the user - **"ENTER THE VALUE OF n :"**. Also read all the 'n' integers from the keyboard one after another at run time by giving the following prompt message - **"NOW ENTER THE INTEGER NUMBERS:"**. Use gedit to enter the program, gcc to compile the program and execute.

Provide screenshots of the program entered, command executed and all the results (you need to try out atleast 30 lists containing more than 16 positive and negative integers).

16. Write a 'C' program to read a list of 'n' integers from the keyboard and count the number of positive numbers, negative numbers, odd numbers, even numbers and prime numbers in the given list and display the results. The value of 'n' must be read from the keyboard at run time by giving the following prompt message to the user - **"ENTER THE VALUE OF n :"**. Also read all the 'n' integers from the keyboard one after another at runtime by giving the following prompt message - **"NOW ENTER THE INTEGER NUMBERS:"**. Use gedit to enter the program, gcc to compile the program and execute. **PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (you need to try out atleast 30 lists containing more than 16 positive, negative, odd, even and prime integers).**

17. Write a 'C' program to read 'n' real numbers (floating point numbers) from the keyboard and compute the sum and average of the real numbers in the list and display the results. **The value of 'n' must be read from the keyboard at run time by giving the following prompt message to the user - "ENTER THE VALUE OF n :". Also read all the 'n' real numbers from the keyboard one after another at run time by giving the following prompt message - "NOW ENTER THE REAL NUMBERS:". Use gedit to enter the program, gcc to compile the program and execute.**
PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (you need to try out atleast 30 lists containing more than 10 positive and negative real numbers).
18. Write a 'C' program to read 'n' names (strings) from the keyboard and rearrange the names in alphabetical order and display the results. The value of 'n' must be read from the keyboard at run time by giving the following prompt message to the user - **"ENTER THE VALUE OF n :". Also read all the 'n' names from the keyboard one after another at run time by giving the following prompt message - "NOW ENTER THE NAMES:". Use gedit to enter the program, gcc to compile the program and execute.**
PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (you need to try out atleast 30 lists containing more than 16 names of different characters).
19. Write a 'C' program to read a sentence containing uppercase characters, lowercase characters, numerals and other special characters from the keyboard and convert all the uppercase letters to lowercase letters and display the results. Use gedit to enter the program, gcc to compile the program and execute.
PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS (you need to try out atleast 30 sentences of different characters).
20. Write a 'C' program to read a sentence containing uppercase characters, lowercase characters, numerals and other special characters from the keyboard and convert all the lowercase letters

in the sentence to uppercase letters and display the results. Use **gedit** to enter the program, **gcc** to compile the program and execute. **PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS** (you need to try out atleast 30 sentences of different characters).

21. Write a 'C' program to read a sentence containing uppercase characters, lowercase characters, numerals and other special characters from the keyboard and count the number of characters, alphabets, numerals and words in the given sentence and display the results. Use **gedit** to enter the program, **gcc** to compile the program and execute. **PROVIDE SCREENSHOTS OF THE PROGRAM ENTERED, COMMAND EXECUTED AND ALL THE RESULTS** (you need to try out atleast 30 sentences of different characters).

PART III : UNIX/LINUX COMMANDS

22. Describe the purpose of the following Unix/Linux commands providing correct syntax(usage format of the command). Use **man** command at the shell to understand the use and purpose of these commands (For example: enter the command **\$man pwd** to get the help to understand **pwd** command).

PROVIDE ONE PAGE OF MAN COMMAND OUTPUT SCREEN SHOT FOR EACH OF THE COMMANDS BELOW.

- | | | | | |
|----------------------|------------------|-------------------|------------------------|----------------|
| i) ls, ls -la | ii) pwd | iii) who | iv) date | v) cat |
| vi) cp | vii) mv | viii) cd | ix) mkdir | x) more |
| xi) less | xii) head | xiii) tail | xiv) ps, ps -la | |
| xv) sudo | xvi) find | xvii) grep | xviii) echo | |
| ix) chmod | xx) free | xxi) df | xxii) du | |
| xxiii) passwd | xxiv) wc | xxv) rm | | |

23. THIS QUESTION MUST BE ATTEMPTED ONLY AFTER COMPLETING PART I & II QUESTIONS.

**EXECUTE THE FOLLOWING COMMANDS ONE AFTER ANOTHER IN THE SAME GIVEN ORDER 1) TO 100),
Examine the output correctly to understand the commands and indicate the outputs generated. Clearly explain the format of the output with all the fields and their meaning.**

PROVIDE ALL THE SCREEN SHOTS OF COMMANDS EXECUTED AND THE OBTAINED OUTPUT RESULTS.

- 1) `echo "HELLO -START OF EXECUTION OF LINUX/UNIX COMMANDS"`
- 2) `PS1="\d \t \u@\h \#\$"`
- 3) `pwd`
- 4) `ls`
- 5) `ls -la`
- 6) `uname -r`
- 7) `uname -a`
- 8) `cat palindrome.c`
- 9) `cat key_search.c`
- 10) `cp sum_avgl.c sa_list.c`
- 11) `cat sa_list.c`
- 12) `cp lower2upper.c l2u.c`
- 13) `cat l2u.c`
- 14) `mkdir spha1`
- 15) `cp l2u.c spha1/f1.c`
- 16) `cp sa_list.c spha1/f2.c`
- 17) `cp key_search.c spha1/f3.c`
- 18) `cd spha1`
- 19) `ls`
- 20) `cat f1.c`
- 21) `mv f2.c f4.c`
- 22) `ls`
- 23) `mv f3.c f5.c`
- 24) `ls`
- 25) `cp f5.c f6.c`
- 26) `cat f6.c`
- 27) `ls`
- 28) `rm f5.c`
- 29) `cat f5.c`
- 30) `ls`

```
31) pwd
32) cd ..
33) pwd && ls spha1
34) chmod 777 palindrome.c
35) ls -l palindrome.c
36) chmod 751 palindrome.c
37) ls -l palindrome.c
38) cat sum_avgl.c key_search.c > progall.c
39) cat progall.c
40) cat palindrome.c lower2upper.c upper2lower.c >> pro-
    gall.c
41) more progall.c
42) less progall.c
43) head -15 progall.c
44) tail -10 progall.c
45) head -1 progall.c
46) tail -1 progall.c
47) cat progall.c | more
48) ls
49) touch emptyfile
50) ls
51) cp key_search.c find.c && ls && cat find.c
52) echo "HELLO GOOD MORNING! HOW ARE YOU"
53) echo "HELLO GOOD MORNING! HOW ARE YOU" | rev
54) echo "MADAM - LIVE ON EVIL" | rev
55) echo $PWD
56) echo $PATH | more
57) wc progall.c
58) wc -c progall.c
59) wc -w progall.c
60) wc -l progall.c
61) find / -name passwd
62) find / -name key_search.c
63) grep "stdio" progall.c
64) grep "scanf" progall.c
65) grep "printf" progall.c
66) grep "for" progall.c
67) grep "uname" progall.c
68) cd /
69) pwd
70) ls && ls -la
71) cd
72) pwd
73) cd ..
74) pwd
```

- 75) **cd**
- 76) **pwd**
- 77) **df -h**
- 78) **du -h**
- 79) **ps**
- 80) **ps -la**
- 81) **date**
- 82) **cal**
- 83) **passwd**
- 84) **who**
- 85) **who -a**
- 86) **whoami**
- 87) **cat >> myteamlist**
[After you press Enter key, cursor will move to the next line and wait for input data. Now enter your team member complete details – Name, Student ID, Section Number & Email ID one after another on separate lines and at the end press Control D (Ctrl & D keys simultaneously) to save the entered text in file myteamlist]
- 88) **ls && cat myteamlist**
- 89) **sort myteamlist**
- 90) **sort -r myteamlist**
- 91) **cat /proc/cpuinfo > infocpu**
- 92) **head -1 infocpu**
- 93) **head -5 infocpu**
- 94) **head -27 infocpu > core0info**
- 95) **cat core0info**
- 96) **tail -5 infocpu**
- 97) **tail -27 infocpu > core4info**
- 98) **cat core4info**
- 99) **history**
- 100) **echo “ END OF SP LAB ASSIGNMENT 1 - GOOD BYE”**
